

Customer Spending Prediction

An easy explanation of the Multiple Linear Regression project

This project predicts how much a customer may spend using age, income, purchase frequency, gender, and education level.

Model: Multiple Linear Regression

Dataset: 1,000 customer records

Tools: Python, pandas, scikit-learn, and pickle

GitHub description: A machine learning project that predicts customer spending from demographic and purchasing behavior data.

Source: the supplied customer spending prediction notebook.

1. What the project does

This is a supervised machine-learning project. The dataset already contains a known spending value for each customer. The model studies the relationship between customer information and spending, then uses that relationship to predict spending for a new customer.

The workflow is:

- Load the customer CSV file.
- Remove columns that are not used for prediction.
- Convert text categories into numeric dummy variables.
- Choose the input columns and the spending target.
- Split the data into training and testing sets.
- Train a LinearRegression model.
- Evaluate the model with R^2 .
- Save the trained model with pickle.
- Collect new customer details and predict spending.

Simple flow: customer data → cleaning → encoding → training → evaluation → saved model → new customer → predicted spending

2. Understanding the dataset

The dataset contains 1,000 rows and eight columns. Each row represents one customer. The original notebook displays the following information:

Column	Meaning	Use
name	Customer name	Removed
age	Customer age	Input
gender	Male or Female	Input
education	Education level	Input
income	Customer income	Input
country	Customer country	Removed
purchase_frequency	Purchase frequency	Input
spending	Known spending amount	Target

The model does not need names or countries in this version. Those columns are removed because they are not included in the selected feature list.

3. Loading and cleaning the data

pandas reads the CSV file into a DataFrame, which is a table-like object that supports filtering, transformation, and analysis.

```
import pandas as pd

dataset = pd.read_csv("customer_data.csv")
dataset = dataset.drop(columns=["name", "country"])
```

pd.read_csv() loads the customer file. drop() removes the name and country columns. The remaining columns contain the information used by the model.

4. Converting text into numbers

A regression model needs numeric inputs. Gender and education are text categories, so the notebook converts them into dummy variables.

```
dataset = pd.get_dummies(dataset, drop_first=True)
```

After encoding, the notebook has these columns:

- **gender_Male**
- **education_High School**
- **education_Master**
- **education_PhD**

The dropped categories become the baselines. Female is the gender baseline because the remaining gender column is gender_Male. Bachelor is the education baseline because the remaining education columns represent High School, Master, and PhD.

For example, a female customer has gender_Male = 0. A customer with a Bachelor degree has all three education dummy columns equal to 0.

5. Selecting features and the target

The model separates the data into **X**, the input features, and **y**, the target value it must predict.

```
x = dataset[
    [
        "age",
        "income",
        "purchase_frequency",
        "gender_Male",
        "education_High School",
        "education_Master",
        "education_PhD",
    ]
]

y = dataset[["spending"]]
```

There are seven input features: age, income, purchase frequency, one gender indicator, and three education indicators. Spending is the target.

6. Splitting the data

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=11,
)
```

The model trains on 80% of the 1,000 rows and is evaluated on the remaining 20%, which is 200 rows. `random_state=11` makes the split repeatable.

7. Training the model

```
from sklearn.linear_model import LinearRegression

regressor = LinearRegression()
regressor.fit(x_train, y_train)
```

`LinearRegression` learns coefficients for the seven features. It attempts to find a line, or more precisely a multi-dimensional plane, that best matches the training examples.

8. Making test predictions

```
y_pred = regressor.predict(x_test)
```

The model predicts spending for the 200 customers in the test set. Those predictions are compared with the real values in `y_test`.

9. Measuring performance with R^2

```
from sklearn.metrics import r2_score

percentage = r2_score(y_test, y_pred) * 100
print(f"Result: {percentage}%")
```

The notebook reports an R^2 score of approximately **93.49%** for this test split. R^2 describes how much variation in spending is explained by the model. It is an evaluation score, not a guarantee for every future customer.

Because the dataset is synthetic or limited and the result depends on the chosen split, the score should be treated as a project evaluation rather than a business promise.

10. Saving the trained model

```
import pickle

filename = "spending.sav"
pickle.dump(regressor, open(filename, "wb"))

loaded_model = pickle.load(open(filename, "rb"))
```

`pickle.dump()` saves the trained model to `spending.sav`. `pickle.load()` loads it again so the prediction program can reuse the model without retraining it.

11. Reading new customer input

```
age = int(input("Enter age: "))
gender = input("Enter gender (Male/Female): ").lower()
education = input(
    "Enter education (High School/Bachelor/Master/PhD): "
).lower()
income = float(input("Enter income: "))
purchase_frequency = float(
    input("Enter purchase frequency: ")
)
```

The input values are converted to numeric types. Age is an integer, while income and purchase frequency are decimal-capable values.

12. Encoding gender and education input

```
if gender == "male":
    gender_male = 1
else:
    gender_male = 0

if education == "high school":
    education_high_school = 1
    education_master = 0
    education_phd = 0
elif education == "master":
    education_high_school = 0
    education_master = 1
    education_phd = 0
elif education == "phd":
    education_high_school = 0
    education_master = 0
    education_phd = 1
elif education == "bachelor":
    education_high_school = 0
    education_master = 0
    education_phd = 0
else:
    print("Invalid education")
    raise SystemExit
```

This recreates the same encoding used during training. Bachelor is the baseline, so it is represented by three zeros. Female is the baseline for gender, so it is represented by `gender_male = 0`.

13. Creating a named prediction row

```
prediction_input = pd.DataFrame(
    [[
        age,
        income,
        purchase_frequency,
        gender_male,
        education_high_school,
        education_master,
        education_phd,
    ]],
    columns=[
        "age",
        "income",
        "purchase_frequency",
        "gender_Male",
        "education_High School",
        "education_Master",
        "education_PhD",
    ],
)
```

The DataFrame uses the same column names and order as the training features. This is safer than passing a plain list and avoids the common scikit-learn warning about missing feature names.

14. Predicting and displaying spending

```
result = loaded_model.predict(prediction_input)
predicted_spending = float(result[0][0])

print(
    f"For a {age} year old {gender.title()} with "
    f"{education.title()} education, spending is "
    f"{predicted_spending:.2f}"
)
```

The model returns an array containing one prediction. `result[0][0]` extracts the number and `float()` converts it into a regular Python value for formatting.

15. Important implementation notes

- The gender code treats every value other than “male” as female. A cleaner version should validate that the user entered Male or Female.
- The education code validates the four expected values and exits for an unknown value.
- A linear regression model can produce a negative prediction for unusual inputs, even though real spending cannot be negative. Input validation or a model such as non-negative regression could be considered later.
- The output example in the notebook predicts 17709.40 for a 52-year-old Female customer with PhD education and the entered income and frequency.

The prediction is an estimate based on patterns in the training data. It should not be treated as a guaranteed amount a customer will spend.

16. Recommended GitHub structure

```
customer-spending-prediction/  
|-- data/  
|   |-- customer_data.csv  
|-- models/  
|   |-- spending.sav  
|-- train_model.py  
|-- predict.py  
|-- requirements.txt  
|-- .gitignore  
|-- customer_spending_prediction_explanation.pdf
```

Upload the dataset, the two Python files, requirements.txt, .gitignore, the saved spending.sav model, and this explanation PDF. Do not upload __pycache__, virtual environments, or secret files.

17. How to run the project

```
pip install -r requirements.txt  
python train_model.py  
python predict.py
```

Run the training script first to create spending.sav. Then run the prediction script and enter the new customer's values.

In one sentence: this project learns from previous customer behavior and uses a linear regression model to estimate the spending of a new customer.